Dor: Smart Queue System

Sprint and Release Plan

Student Affairs Office, University | Prepared by Eman

1. Summary

Dor replaces paper queue numbers at the Student Affairs office. A student takes a digital ticket on their phone and follows their place in line, staff call students to their desks, and a waiting-room screen shows who is being served.

The work is split into two **sprints** (fixed work cycles) of 10 working days each. Each sprint ends with a **release**: a complete, working version of the product that can be demonstrated and used.

	Sprint 1	Sprint 2
Days	Day 1 to Day 10	Day 11 to Day 20
Working time	10 days, 8 hours a day	10 days, 8 hours a day
Flexible day	Day 9: Should work that can move to Sprint 2	Day 17: Could work that can be dropped
Ends with	Release 1: a working queue system	Release 2: live and complete system

2. Planning approach

- **Complete features, one at a time.** Each feature is built across the whole application (server, user interface, and tests) before the next one starts. At any point, everything finished so far works end to end.
- **Each release is a full product.** Release 1 does not include every feature, but every feature it includes is finished, tested, and usable.
- **Planned day by day.** Each day focuses on one main feature, sized to an 8-hour working day. Each sprint has one flexible day, filled with lower-priority work that can move or be dropped if earlier days run late.
- **Fixed scope during a sprint.** New ideas go to the next sprint, not the current one.

Feature priorities (MoSCoW)

Each feature has a priority from the MoSCoW method, which ranks features by how necessary they are for the release:

Priority	Meaning in this plan
Must have	The release does not work without it. Never cut.
Should have	Important, but the release still works without it. Cut only if the sprint falls behind.
Could have	Useful extra. The first to be cut if time runs short.
Won't have	Agreed to be outside both releases, and recorded for a later phase.

If a sprint falls behind, Could items are cut first, then Should items. Must items are never cut.

Definition of Done

A feature counts as done only when:

- Its server part and its interface are both complete.
- Its automated tests pass.
- It is merged through a reviewed pull request.
- It works in the full application, not only in isolation.

3. Sprint 1: Release 1, a working queue

Sprint goal: A student logs in, takes a ticket, and follows their place in line. Staff call and serve students, and two desks never receive the same student. Screens refresh automatically every few seconds.

Each day focuses on one main feature. The order follows the dependencies: accounts come first because every screen needs a signed-in user, services come before tickets because a ticket belongs to a service, and the staff desk comes after tickets because it serves them. The lower-priority work sits on Day 9, so it can move to Sprint 2 if earlier days run late.

Day	Feature and tasks	Priority
Day 1	**Setup and design** • Write the Sprint 1 requirements and user stories • Design the database tables and the relations between them • Define the server endpoints (the addresses the interface calls) for each feature • Create the repository, the server and interface projects, and the database • Check that the server, interface, and database run together	Must
Day 2	**Accounts and login: server** • Users with three roles: student, staff, and admin • Registration and login, with passwords stored encrypted • Issue a login token (a signed proof that the user is signed in) and check it on every request • Restrict each endpoint to the roles allowed to use it	Must
Day 3	**Accounts and login: tests and interface** • Tests for login and access rules • Login and registration pages • Keep the user signed in, and send the token with each request • Send each role to its own screen after login	Must
Day 4	**Services and counters** • Server: create, edit, and activate or deactivate services and counters • Admin screen to manage services and counters • Tests for services and counters	Must

Day	Feature and tasks	Priority
Day 5	**Take a ticket: server** • Ticket numbers restart each day for each service • A student can hold only one active ticket at a time • Calculate the student's position in line and the estimated wait • Tests for numbering, the one-ticket rule, position, and estimated wait	Must
Day 6	**Take a ticket: student screen** • Choose a service and take a ticket • See the ticket number, position, and estimated wait • Screen refreshes automatically every few seconds	Must
	Leave the queue • Server rule, a button on the student screen, and tests	Should
Day 7	**Staff desk: server** • Call next: give the desk the student who has waited the longest • Mark the student done or no-show • Allow only valid status changes, and reject others with a clear message • Protect against two desks calling the same student at the same moment • Tests, including two desks calling at once	Must
Day 8	**Staff desk: interface** • Staff screen: choose a desk, call next, mark done or no-show • Show the current student, and enable only the valid buttons • Show a clear message when an action is rejected	Must
	End-to-end testing • Run the full flow: student, staff, and admin together • Record every bug found	Must
Day 9	**Bug fixes** • Fix the bugs found on Day 8, and repeat the full-flow test	Must
	Waiting-room screen • Public screen with no login: who is being served at each desk and who is next • Refreshes automatically, with tests • Moves to Sprint 2 if earlier days run late	Should
Day 10	**Release 1** • API documentation and a README on how to run and use the system • Check every feature against the Definition of Done • Publish Release 1 • Sprint review and demo with the mentor, then a short retrospective	Must

Release 1 delivers

- A complete flow for students, staff, and the admin, plus the waiting room if it fits.
- Automated tests for every feature, including two desks calling at the same time.
- Documentation and a demo of the full flow.

4. Sprint 2: Release 2, live and complete

Sprint goal: Every screen updates instantly when the queue changes. Students who are far away get an earlier alert. The admin manages user roles.

Live updates come first and get three days, because they are the main change and the newest technology. Should features follow, one per day. The Could features sit on Day 17, so they are dropped first if time runs short. Testing and the release come last, after all features are in.

Day	Feature and tasks	Priority
Day 11	**Instant live updates: server** • Open a live connection between the server and the screens • Check the login token when a screen connects • Send the new queue status after every change • Tests that changes reach connected screens	Must
Day 12	**Instant live updates: student and staff screens** • Student and staff screens update instantly • Reconnect automatically if the connection drops	Must
Day 13	**Instant live updates: waiting room and tests** • Waiting-room screen updates instantly • Replace the refresh-every-few-seconds approach from Sprint 1 • Tests for instant updates and reconnecting	Must
Day 14	**Earlier alert for students who are far away** • Student chooses near, medium, or far when taking a ticket • Alert the student earlier when they are far • Tests for the alert	Should
Day 15	**Estimated wait from real service times** • Record how long each service takes • Calculate the average from recent tickets, with a default value at the start of the day • Show the updated estimated wait, with tests	Should
Day 16	**Admin assigns user roles** • Server: admin changes a user's role, with access rules • Admin screen to view users and change their roles • Tests for role changes	Should

Day	Feature and tasks	Priority
Day 17	**Admin daily statistics** • Tickets served, no-shows, and average wait for the day, on an admin screen	Could
	Automatic no-show after 2 minutes • Mark a called student as no-show automatically after 2 minutes	Could
Day 18	**Edge cases and security review** • Review the access rules on every endpoint and live connection • Test unusual cases: actions out of order, repeated clicks, dropped connections • Fix the problems found	Must
Day 19	**Regression testing and documentation** • Retest every feature from Sprint 1 and Sprint 2, and fix the bugs found • Update the API documentation and the README • Prepare the final demo script	Must
Day 20	**Release 2** • Check every feature against the Definition of Done • Publish Release 2 • Final demo with the mentor, then a short retrospective	Must

5. Reviews and check-ins

When	What happens
Day 1 and Day 11	Sprint planning: confirm the features, estimates, and sprint goal
End of each day	Short check-in: what I finished, what comes next, any blocker
Day 5 and Day 15	Mid-sprint check: compare progress with the plan, and adjust if needed
Day 10 and Day 20	Sprint review and release demo with the mentor, then a short retrospective